

CM 4 - DICTIONNAIRES ET SETS

PROGRAMMATION PYTHON – MASTER 1 MAS

Romain Tavenard

2026

✉ romain.tavenard@univ-rennes2.fr



LES DICTIONNAIRES



3 parcours de dictionnaires en Python :

```
mon_dico = {"a": 4, "c": 3, "z": 3}

for val in mon_dico.values(): # Parcours par valeur
    print(val)

for cle in mon_dico.keys(): # Parcours par clé
    print(cle)

for cle, val in mon_dico.items(): # Parcours par clé ET valeur
    print(cle, valeur)
```

3 parcours de dictionnaires en Python :

```
mon_dico = {"a": 4, "c": 3, "z": 3}

for val in mon_dico.values(): # Parcours par valeur
    print(val)

for cle in mon_dico.keys(): # Parcours par clé
    print(cle)

for cle, val in mon_dico.items(): # Parcours par clé ET valeur
    print(cle, valeur)
```

Question subsidiaire : que vaut x dans le parcours suivant ?

```
for x in mon_dico:
    print(x)
```

On dispose d'un dictionnaire $d = \{ "a": 4, "c": 3, "z": 3 \}$.

Quel parcours utiliser pour...

- Déterminer le minimum des valeurs du dictionnaire d ?

On dispose d'un dictionnaire $d = \{ "a": 4, "c": 3, "z": 3 \}$.

Quel parcours utiliser pour...

- Déterminer le minimum des valeurs du dictionnaire d ? **Par valeur**
- Déterminer la clé correspondant au minimum des valeurs du dictionnaire d ?

On dispose d'un dictionnaire $d = \{ "a": 4, "c": 3, "z": 3 \}$.

Quel parcours utiliser pour...

- Déterminer le minimum des valeurs du dictionnaire d ? **Par valeur**
- Déterminer la clé correspondant au minimum des valeurs du dictionnaire d ? **Par clé ET valeur**
- Créer un nouveau dictionnaire défini comme $\{ "a": f(4), "c": f(3), "z": f(3) \}$?

On dispose d'un dictionnaire $d = \{ "a": 4, "c": 3, "z": 3 \}$.

Quel parcours utiliser pour...

- Déterminer le minimum des valeurs du dictionnaire d ? **Par valeur**
- Déterminer la clé correspondant au minimum des valeurs du dictionnaire d ? **Par clé ET valeur**
- Créer un nouveau dictionnaire défini comme $\{ "a": f(4), "c": f(3), "z": f(3) \}$? **Par clé ET valeur**

```
max_val = PAS_BEaucoup_DU_TOUT # Mais quoi ?  
for val in mon_dico.values():  
    if val > max_val:  
        max_val = val  
# À la fin de la boucle, la valeur maximale est stockée dans `max_val`
```

```
max_val = PAS_BEUCOUP_DU_TOUT # Mais quoi ?
for val in mon_dico.values():
    if val > max_val:
        max_val = val
# À la fin de la boucle, la valeur maximale est stockée dans `max_val`
```

```
max_val = PAS_BEUCOUP_DU_TOUT # Mais quoi ?
cle_max = None
for cle, val in mon_dico.items():
    if val > max_val:
        max_val = val
        cle_max = cle
# À la fin de la boucle, l'argmax est stocké dans `cle_max`
```

Un dictionnaire peut avoir pour clés :

- des chaînes de caractères
- des entiers
- des flottants (mais c'est une mauvaise idée)
- des tuples (ex : (1, 3))
- **mais PAS des listes ni des dictionnaires**

Si l'on souhaite accéder à la valeur associée à la clé `"langue"`, pas besoin de parcourir !

Si l'on souhaite accéder à la valeur associée à la clé "langue", pas besoin de parcourir !

```
prefs = {"theme": "sombre", "taille_police": 14}
```

```
# Option 1
```

```
if "langue" in prefs:
```

```
    langue = prefs["langue"]
```

```
else:
```

```
    langue = "fr"
```

```
# Option 2 (À PRIVILÉGIER)
```

```
langue = prefs.get("langue", "fr")
```

“Écrire une fonction qui prend en entrée une chaîne de caractère et retourne le caractère le plus fréquent de cette chaîne.”

Comme pour les listes, on devra prendre garde aux pièges suivants :

- On ne doit **jamais** parcourir un dictionnaire pendant qu'on lui insère / supprime des paires clés-valeurs ;
- Si l'on souhaite effectuer une copie de dictionnaire, il est bon d'utiliser la fonction `dict()` (se référer au module `deepcopy` si besoin de copies profondes).

“ On a vu dans le cours précédent qu’une matrice peut être représentée comme une liste de listes. Proposez une structure de données plus adaptée pour représenter les matrices creuses (c’est-à-dire les matrices dans lesquelles de nombreuses valeurs valent 0), comme dans l’exemple suivant :

$$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

Comment faut-il faire pour accéder à l’élément d’indice (i, j) d’une matrice ainsi représentée ? ”

LES SETS



Un set peut être vu comme un **dictionnaire sans valeurs** (que des clés).

Quelle utilité ?

- stocker des ensembles sans répétition
- construction simple à partir de liste ou chaîne :

```
set([1, 3, 2, 1]) # {1, 3, 2}
set("abcdcb")    # {"a", "b", "c", "d"}
```

- méthodes sur les sets :
 - insertion : `mon_set.add(item)`
(si doublon, pas d'insertion)
 - union : `{"a", "b"} | {"b", "c"}` ou
`{"a", "b"}.union({"b", "c"})`
 - intersection : `{"a", "b"} & {"b", "c"}` ou
`{"a", "b"}.intersection({"b", "c"})`

“ Écrire une fonction qui prend en entrée une chaîne de caractère et retourne le caractère le plus fréquent de cette chaîne (sans utiliser de dictionnaire).

”

STRUCTURES IMBRIQUÉES



En pratique, les vraies données sont souvent du type :

```
[
  {
    "address": {
      "building": "1449",
      "street": "Nostrand Avenue",
      "zipcode": "11226"
    },
    "grades": [
      {"date": "2014-10-21T02:00:00", "grade": "B"},
      # [...]
      {"date": "2011-12-14T01:00:00", "grade": "A"}
    ],
    "name": "Nostrand Donut Shop"
  },
  # [...]
]
```

Pour visualiser :

```
from pprint import pprint  
  
pprint(ma_structure_compliquee)
```

Pour accéder aux éléments :

```
pprint(ma_structure_compliquee[0]) # Visualiser un niveau,  
                                   # comprendre son type
```

Pour visualiser :

```
from pprint import pprint

pprint(ma_structure_compliquee)
```

Pour accéder aux éléments :

```
pprint(ma_structure_compliquee[0]) # Visualiser un niveau,  
                                   # comprendre son type
```

```
pprint(ma_structure_compliquee[0]["address"]) # Visualiser un niveau,  
                                                # comprendre son type
```

“Écrivez un morceau de code qui permet de stocker l'ensemble des notes ("A", "B", etc.) attribuées (sans doublon) dans le jeu de données ci-dessous.

”

```
[
  {
    "address": {"building": "1449", "street": "Nostrand Avenue"},
    "grades": [
      {"date": "2014-10-21T02:00:00", "grade": "B"},
      # [...]
      {"date": "2011-12-14T01:00:00", "grade": "A"}
    ],
    "name": "Nostrand Donut Shop"
  },
  # [...]
]
```